

1. Presentación del problema

El objetivo de este documento es presentar un algoritmo de priorización de pedidos de una tienda online basándose en los siguientes factores:

1. **priorityScore**: Calculado anteriormente con factores externos. int: [0, 100].
2. **dispatchWindow**: Tiempo restante para el vencimiento. int: [-inf, 0), (0, inf].
3. **sizeCategory**: Tamaño del pedido. dict: {P: int, M: int, G: int}

2. Algoritmo: DynaHeap

Este algoritmo está motivado por una serie de suposiciones operativas:

- No conocemos límites de espacio o tiempos de despacho
- Contamos con tres variables de distinto tipo que definen la prioridad del envío
- El único dato final que se necesita es la siguiente orden para despachar
- Los pedidos pequeños son más comunes y fáciles de enviar

Dado que sólo necesitamos leer un dato a la vez, se eligió un Max Heap como método de cola de prioridad principal, y el valor en el que se basa es un puntaje combinado dinámico calculado del siguiente modo:

$$W_1 \cdot \text{priorityScore} + W_2 \cdot \frac{1}{\text{dispatchWindow}} + \text{sizeBonus}, \quad \text{dispatchWindow} > 0$$

$$W_1 \cdot \text{priorityScore} + \text{sizeBonus} + W_3(-\text{dispatchWindow}), \quad \text{dispatchWindow} < 0$$

Teniendo W_n pesos calculados dinámicamente, como se presentará más adelante. Contamos con tres tipos de variables, y el modo en el que se manejan depende de su naturaleza:

- **priorityScore** se multiplica por un número real: es proporcional al puntaje combinado. Responde a la prioridad del negocio.
- **dispatchWindow**, al ser inversamente proporcional (menos tiempo representa más urgencia; pedidos vencidos aún más), cuenta con dos casos:
 - Pedido a tiempo (> 0): W_2 se multiplica por el inverso de **dispatchWindow**. La naturaleza de $\frac{1}{x}$ hará que se prioricen agresivamente los valores pequeños.
 - Pedido retratado (< 0): W_3 se multiplica por el negativo de **dispatchWindow**. Un valor grande disparará linealmente la urgencia.
- **sizeBonus** nace de la naturaleza categórica del tamaño del pedido: se priorizan los pedidos más pequeños (asumimos que son más fáciles de enviar) con un bono. Evita que se acumulen muchos pedidos pequeños—más comunes.

Este puntaje, al depender del tiempo, se calcula cada 3 minutos (elegido arbitrariamente). Al momento de ingresar un nuevo pedido este entra a una lista de espera, y el cálculo del puntaje se hace simultáneamente con nuevos y viejos pedidos pendientes.

Presentamos un pseudocódigo básico de implementación:

```

FUNCTION CalculateScore(order):
    IF order.dispatchWindow > 0:
        RETURN (W1 * order.priorityScore) + (W2 * (1/order.dispatchWindow)) + order.sizeBonus
    ELSE:
        RETURN (W1 * order.priorityScore) + order.sizeBonus + (W3 * ABS(order.dispatchWindow))
    LogExpired(order.id, CurrentTime())

FUNCTION ProcessThreeMinuteUpdate():
    Move NewOrdersBuffer TO MainList
    FOR EACH order IN MainList:
        order.dispatchWindow -= 3
        IF order.dispatchWindow == 0:
            order.dispatchWindow = -1
        order.score = CalculateScore(order)
    MainHeap = Heapify(MainList)

FUNCTION DispatchNextOrder():
    IF MainHeap IS NOT EMPTY:
        bestOrder = MainHeap.ExtractMax()
    
```

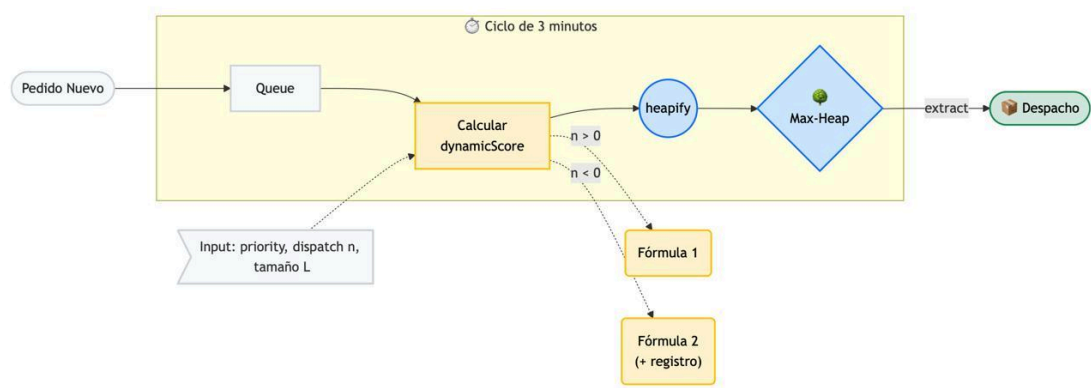
```
DeliverPackage(bestOrder)
LogDispatched(order.id, CurrentTime)
```

Es importante mencionar la consideración de condiciones de carrera: en una implementación real es necesario tener siempre un Heap siempre disponible, y actualizar en segundo plano. Adicionalmente, tenemos las siguientes métricas de rendimiento para optimizar:

- Pedidos vencidos: `dispatchWindow < 0`
- Tiempo de atraso: `dispatchTime - expiredTime`
- Throughput: `dispatchTime`
- Tiempo promedio de despacho: `dispatchTime - insertionTime`

El despacho, en tiempo, cuesta $O(\log n)$ y la inserción—heapify— $O(n)$. La inserción se hace en lotes cada 3 minutos y el cálculo del puntaje es de tiempo constante. Es eficiente mientras sólo sea necesario extraer la raíz del árbol.

3. Diagrama



Un diagrama más detallado se puede encontrar en [este enlace \[click\]](#).

4. Mapa de tradeoffs

TRADE-OFF	
PRO	
EXTRACCIÓN RÁPIDA	Max-Heap da $O(\log N)$ al extraer. El trabajador nunca espera.
FLEXIBILIDAD DEL NEGOCIO	Cambiar pesos ($W1, W2, W3$) ajusta la estrategia sin rehacer el sistema.
EFICIENCIA POR BATCHING	Recalcular cada 3 min con heapify $O(N)$ mantiene CPU baja.
CONTRA	
VISIBILIDAD PARCIAL	El heap solo da el top 1. Ver los próximos 50 pedidos es costoso.
SENSIBILIDAD EXTREMA A PARÁMETROS MATEMÁTICOS	Un error en pesos ($W2$) puede hacer que VIPs sean ignorados.
RIESGO RESIDUAL DE INANICIÓN	Pedidos grandes y de baja prioridad pueden quedar atrapados si llegan muchos pequeños urgentes.

5. Próximos pasos: Implementación, aprendizaje automático, pruebas A/B

El modelo, hasta este punto, se encuentra en su estado más generalizado: sus pesos—que definen el orden final—no se encuentran definidos. Sin embargo, esta es una ventaja: es posible desarrollar un método de optimización. En el repositorio en el cual se encuentra el presente documento se llevó a cabo la optimización con los siguientes parámetros. Esta implementación de búsqueda en cuadrícula fue programada por Deepseek bajo nuestros parámetros. Los resultados se pueden encontrar en [este enlace \[click\]](#).